

IF231303 Arsitektur Perangkat Lunak

Pertemuan 04: Ports and Adapters

September 6, 2026

Alfa Yohannis



Tujuan Pembelajaran

- 1 Menjelaskan peran port dan adapter serta arah ketergantungan yang membuat logika inti bebas dari teknologi penyimpanan.
- 2 Membedakan port masukan dari port keluaran.
- 3 Mengukur kemandirian logika inti, biaya penggantian adapter, dan cakupan pengujian tanpa infrastruktur.

- Bab 2 menjaga arah panggilan tetap ke bawah, dan aturan itu menahan perubahan di satu lapisan.
- Namun lapisan bisnis tetap memanggil lapisan persistensi.
- Artinya logika inti tetap mengenal keberadaan penyimpanan.

Ports and adapters membalik arah ketergantungan. Logika inti tidak lagi memanggil penyimpanan, melainkan menyatakan kebutuhannya sebagai port, lalu pihak lain yang menyediakannya.

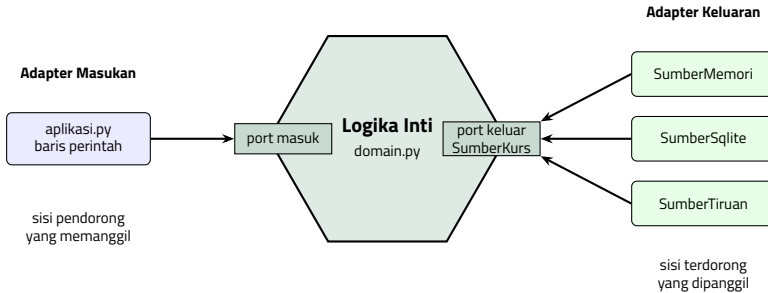
Pembalikan itu menghasilkan satu klaim yang menyebut angka pasti, sehingga klaimnya dapat dibuktikan salah.

Masalah yang Tersisa dari Bab 2

- Layered menjaga arah panggilan tetap ke bawah.
- Aturan itu menahan perubahan di satu lapisan.
- Namun lapisan bisnis tetap memanggil lapisan persistensi.
- Artinya logika inti tetap mengenal keberadaan penyimpanan.

Ports and adapters menyelesaikannya dengan membalik arah ketergantungan.

Bentuk Heksagonal



Adapter berada di kedua sisi. Satu port dapat diisi banyak adapter, tetapi hanya satu yang aktif saat program berjalan.

Masuk dan Keluar Menunjuk Arah Panggilan

- Kata masuk dan keluar menunjuk **arah panggilan**, bukan arah data. Data pada keduanya tetap mengalir dua arah.
- Port masukan berarti panggilannya datang dari luar. Port keluaran berarti panggilannya berangkat dari logika inti.
- Port keluaran karena itu tidak terbatas pada pembacaan. Port penyimpanan lazimnya memuat *simpan* dan *cari* sekaligus.
- Setiap sumber menuntut adapternya sendiri, sedangkan satu adapter hanya mengisi satu port.

Port Ditulis oleh Logika Inti

```
class SumberKurs(Protocol):  
    # Port keluaran yang dibutuhkan logika inti. Siapa pun yang menyediakan  
    # metode ini dapat dipasang, baik memori, basis data, maupun tiruan.  
  
    def nilai(self, kode_asal, kode_tujuan):  
        # Mengembalikan nilai kurs, atau None bila pasangannya tidak ada.
```

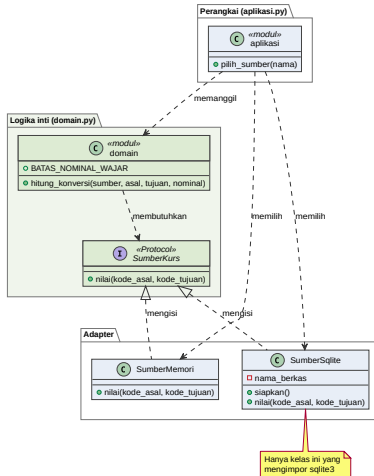
Port hanya menyebut kelakuan yang dibutuhkan, bukan teknologinya.

Adapter Diterima sebagai Argumen

```
def hitung_konversi(sumber, kode_asal, kode_tujuan, nominal):  
    # Sumber kurs diterima sebagai argumen, bukan dibuat di dalam fungsi.  
    # Karena itu fungsi ini tidak pernah tahu teknologi penyimpanannya.  
    if nominal <= 0:  
        raise ValueError("Nominal harus lebih besar dari nol")  
    kurs = sumber.nilai(kode_asal, kode_tujuan)  
    if kurs is None:  
        raise LookupError("Kurs tidak tersedia")  
    return nominal * kurs
```

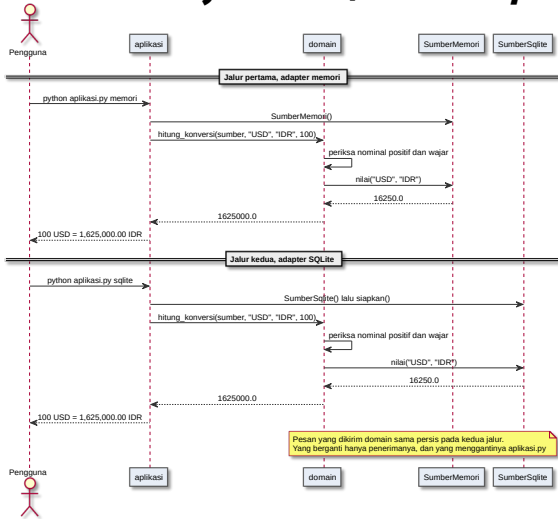
Bila logika inti membuat adapternya sendiri, seluruh manfaatnya hilang.

Diagram Kelas Port dan Adapter



- Port berada di dalam kotak hijau, bersama logika intinya. Letak itulah yang membalik arah ketergantungan.
- Kedua adapter menunjuk ke atas menuju port, dan tidak ada panah yang keluar dari kotak hijau.
- Hanya aplikasi yang mengenal ketiganya sekaligus.
- SumberSqlite punya metode tambahan `siapkan`. Adapter boleh menyediakan lebih dari yang diminta port.

Dua Penjalanan, Pesan yang Sama



- Logika inti mengirim pesan yang sama persis, yaitu nilai("USD", "IDR").
- Nilai balikkannya pun sama, yaitu 16250.0.
- Yang berganti hanya penerimanya, dan penggantinya aplikasi.py.
- Pemanggilan siapkan hanya ada di jalur kedua, dan dijalankan berkas perangkat, bukan logika inti.

Analogi: Stopkontak di Dinding

- Stopkontak menentukan bentuk colokan yang diterimanya.
- Stopkontak tidak menentukan alat apa yang dicolokkan.
- Kipas angin, lampu, dan pengisi daya sama-sama dapat dipasang.
- Dinding tidak perlu diubah setiap kali alatnya berganti.

Port adalah stopkontaknya, adapter adalah colokannya. Logika inti tidak boleh membuat adapternya sendiri, sebab dinding yang memilih alat bukan lagi stopkontak.

Kasus: Klaim yang Gugur karena Satu Baris

- 1 Tim menyusun domain, port, dan dua adapter dengan rapi.
- 2 Muncul kebutuhan mencatat waktu pembacaan ke berkas log.
- 3 Seseorang menambahkan satu baris impor pustaka basis data di `domain.py` agar cepat selesai.
- 4 Susunan berkasnya masih terlihat rapi, dan tidak ada galat muncul.
- 5 Pemeriksaan otomatis melaporkan satu adapter disebut, dan klaimnya gugur.

Karena itulah klaim yang menyebut angka lebih berguna daripada klaim yang hanya menyebut sifat.

Tiga Akibat yang Dapat Diukur

Akibat	Angka	Cara mengukur
Logika inti tidak mengenal teknologi	0 adapter	Menghitung modul yang di-impor domain.py
Penggantian adapter tidak menyentuh logika inti	1 baris	Mengganti pilihan pada berkas perangkat
Pengujian berjalan tanpa in-frastruktur	0 basis data	Memasang adapter tiruan berisi satu metode

Klaim yang menyebut angka dapat dibuktikan salah. Itu yang membuatnya berguna.

- Logika inti menyatakan kebutuhannya sebagai port, lalu adapter mengisinya. Seluruh panah menunjuk ke dalam.
- Satu port dapat diisi banyak adapter, dan keduanya menghasilkan jawaban yang sama tanpa mengubah logika inti.
- Adapter diserahkan dari luar sebagai argumen, dan pemilihannya dipusatkan pada satu berkas perangkat.
- Klaim utamanya menyebut angka pasti, yaitu nol teknologi penyimpanan yang dikenal logika inti.
- Harganya adalah jumlah berkas yang bertambah, sehingga belum tentu sepadan untuk aplikasi kecil.

Persiapan Latihan

```
# sekali saja, dari akar repositori
bash sources/siapkan_venv.sh
source .venv/bin/activate

cd sources/chapter04
python aplikasi.py memori
```

Venv yang sama dipakai kembali, sehingga cukup diaktifkan. Satu logika inti, dua adapter, tiga masalah berbeda. Mengganti penyimpanan berarti mengganti satu baris pada `aplikasi.py`, tanpa menyunting `domain.py` sama sekali.

Latihan 1: Membuktikan Kemandirian

```
python periksa_port.py
```

- 1 Catat baris Adapter disebut.
- 2 Buka `domain.py`, lalu pastikan tidak ada nama teknologi penyimpanan di dalamnya.
- 3 Tambahkan `import sqlite3` pada `domain.py`, lalu jalankan ulang pemeriksaannya.
- 4 Batalkan penambahan tersebut, lalu pastikan pemeriksaan kembali bersih.

Pertanyaan: satu modul standar tetap diimpor. Mengapa impor itu tidak melanggar klaim?

Latihan 2: Mengukur Biaya Penggantian

```
python aplikasi.py memori  
python aplikasi.py sqlite
```

- 1 Pastikan hasil kedua adapter sama.
- 2 Catat berkas mana saja yang harus dibuka untuk berpindah adapter.
- 3 Tulis satu adapter ketiga yang membaca kurs dari berkas teks biasa.
- 4 Catat jumlah berkas dan baris yang berubah agar adapter ketiga dapat dipakai.

Pertanyaan: apakah `domain.py` ikut berubah ketika adapter ketiga ditambahkan?

Latihan 3: Menguji Tanpa Infrastruktur

```
python uji_tanpa_basis_data.py
```

- 1 Catat hasil ketiga pengujiannya.
- 2 Hitung jumlah baris kelas SumberTiruan.
- 3 Tambahkan satu pengujian baru untuk nominal yang melewati batas wajar.
- 4 Hitung berapa banyak infrastruktur yang disentuh seluruh pengujian.

Pertanyaan: berapa baris kode yang dibutuhkan untuk menggantikan seluruh basis data pada pengujian?